

Project Documentation: FastAPI To-Do List CRUD API

1. Project Goals

The primary goal of this project was to build a simple, lightweight RESTful API for managing a To-Do list using FastAPI and Python. This serves as a foundational exercise to understand API routing, HTTP methods, status codes, and data validation without the complexity of a persistent database.

2. What Was Done

- Initialized a Python virtual environment and Git repository.
- Created a FastAPI application (main.py) with an in-memory database (tasks_db).
- Implemented full CRUD operations (Create, Read, Update, Delete) with appropriate status codes (200, 201, 204).
- Added robust error handling (400 Bad Request for invalid input, 404 Not Found for missing tasks).
- Implemented optional query parameters for filtering tasks by status and searching by title.
- Created utility endpoints for health checks (/health), statistics (/stats), and resetting the database (/reset).
- Simulated an "AI vs Me" code review by generating an alternative implementation (ai-version/main.py) to compare design choices.
- Wrote a comprehensive README.md and executed a 7-stage Git commit history.

3. Features, Uses, and "Why & How"

A. In-Memory Storage (tasks_db)

What it is: A simple Python list pre-filled with 3 default tasks.

Use/Goal: To store task data temporarily while the server runs.

Why: It removes the overhead of configuring SQL or NoSQL databases, making it perfect for rapid prototyping and learning.

How: Stored globally as tasks_db: List[Task]. Note that due to the "Mortality Experiment", this data is wiped clean whenever the server restarts.

B. Pydantic Data Models

What it is: TaskInput and Task classes inheriting from BaseModel.

Use/Goal: To enforce strict data types (e.g., ensuring 'id' is an integer and 'done' is a boolean).

Why: To automatically validate incoming JSON requests and format outgoing responses, preventing bad data from crashing the app.

How: Passed into the endpoint functions as type hints. FastAPI automatically uses them to parse requests.

C. Core CRUD Endpoints

1. POST /tasks (Create): Accepts a title. Validates it isn't empty. Calculates the next ID and appends it to the list. Returns **201 Created**.

2. GET /tasks (Read All): Returns the list of tasks. Includes optional query parameters ?done=true and ?search=keyword to filter the list dynamically.

3. GET /tasks/{id} (Read One): Retrieves a specific task by its URL path ID. Returns **404 Not Found** if missing.

4. PUT /tasks/{id} (Update): Modifies the title and completion status of an existing task. Blocks empty titles (400 Bad Request).

5. DELETE /tasks/{id} (Delete): Removes a task from the list and returns **204 No Content** to indicate successful deletion without a response body.

D. Utility Endpoints

1. GET /stats: Calculates and returns the total number of tasks, completed tasks, and open tasks. Useful for dashboard metrics.

2. POST /reset: Clears the current list and restores the original 3 dummy tasks. Essential for quickly restarting tests after the database gets messy.

E. Interactive Documentation (Swagger UI)

What it is: An automatic webpage hosted at /docs.

Use/Goal: To visually explore and interact with the API.

Why: It saves developers from having to use external tools like Postman or write long terminal curl commands to test their endpoints.

How: FastAPI reads the Python code, type hints, and Pydantic models to instantly generate an OpenAPI schema in the background.

Generated automatically for the CRUD API Project Assignment.